



Modul Pemrograman Mobile Lanjut

S1 Ilmu Komputer & D3 Manajemen Informatika

Pertemuan 5:

Adaptive & Responsive Design & Theming

Disusun Oleh:

Asisten Dosen Pemrograman Mobile Lanjut
Jurusan Ilmu Komputer Universitas Lampung

Tim Penyusun

Pengarah

Muhaqiqin, S.Kom., M.T.I.

Agung Pambudi, M.Kom.

Penyusun

Asdos Mola S1

- Dhimas Dzulfajri 2317051076
- Ananda Anhar Subing 2317051082
- Dimas Adivia 2217051075

Asdos Mola D3MI

- Muhammad Ilham Bintang F. E 2357051023
- Fajriyah Indriani 2317051016

Gambaran Umum

Materi pertemuan ini membahas cara membangun antarmuka Flutter yang dapat menyesuaikan diri terhadap ukuran layar dan orientasi perangkat, serta penerapan Material Design dan sistem tema. Praktikum menggabungkan MediaQuery, LayoutBuilder, OrientationBuilder, ThemeData, ColorScheme, dan TextTheme dalam sebuah aplikasi responsive yang memiliki Light Mode dan Dark Mode.

Capaian Pembelajaran

1. Mahasiswa mampu menjelaskan konsep responsive UI, adaptive UI, constraints, ukuran layar, dan orientation pada Flutter.
2. Mahasiswa mampu menggunakan MediaQuery dan LayoutBuilder untuk membuat tampilan yang menyesuaikan ukuran layar.
3. Mahasiswa mampu menerapkan Material Design, ThemeData, ColorScheme, dan TextTheme pada aplikasi Flutter.
4. Mahasiswa mampu membuat Light Theme dan Dark Theme serta menguji tampilan aplikasi pada beberapa kondisi layar.

1. Responsive UI

Responsive UI merupakan pendekatan untuk membuat tampilan aplikasi yang dapat menyesuaikan susunan dan ukuran elemen ketika ruang layar berubah. Pada Flutter, penyesuaian dilakukan dengan memanfaatkan informasi ukuran layar dan constraints yang diberikan oleh parent widget.

Dalam praktikum ini, responsive UI diterapkan pada halaman profil sederhana. Pada ruang yang sempit, informasi disusun secara vertikal. Pada ruang yang lebih lebar, informasi dapat disusun secara horizontal.

Contoh konsep layout responsive



```
1 Widget build(BuildContext context) {
2   return LayoutBuilder(
3     builder: (context, constraints) {
4       if (constraints.maxWidth >= 600) {
5         return const Row(
6           children: [
7             Expanded(child: ProfileHeader()),
8             Expanded(child: ProfileInfo()),
9           ],
10        );
11      }
12
13      return const Column(
14        children: [
15          ProfileHeader(),
16          ProfileInfo(),
17        ],
18      );
19    },
20  );
21 }
```

Catatan: nilai 600 pada contoh digunakan sebagai batas latihan.

Batas tersebut dapat disesuaikan dengan kebutuhan desain dan ruang yang tersedia.

2. Adaptive UI

Adaptive UI menentukan bentuk atau susunan antarmuka yang digunakan berdasarkan kondisi ruang dan perangkat. Contohnya, halaman dapat menggunakan **Column** pada layar sempit dan **Row** pada layar yang lebih lebar.

- Layar sempit: komponen ditampilkan bertumpuk secara vertikal.
- Layar lebih lebar: komponen dapat ditampilkan berdampingan.
- Perubahan layout dilakukan berdasarkan ruang yang tersedia, bukan berdasarkan nama perangkat.

3. Screen Size dan MediaQuery

MediaQuery digunakan untuk mendapatkan informasi mengenai kondisi layar yang sedang digunakan. Pada praktikum ini, MediaQuery digunakan untuk membaca ukuran layar. Informasi ukuran layar tersebut dapat digunakan untuk menyesuaikan padding, ukuran teks, atau susunan elemen pada halaman.

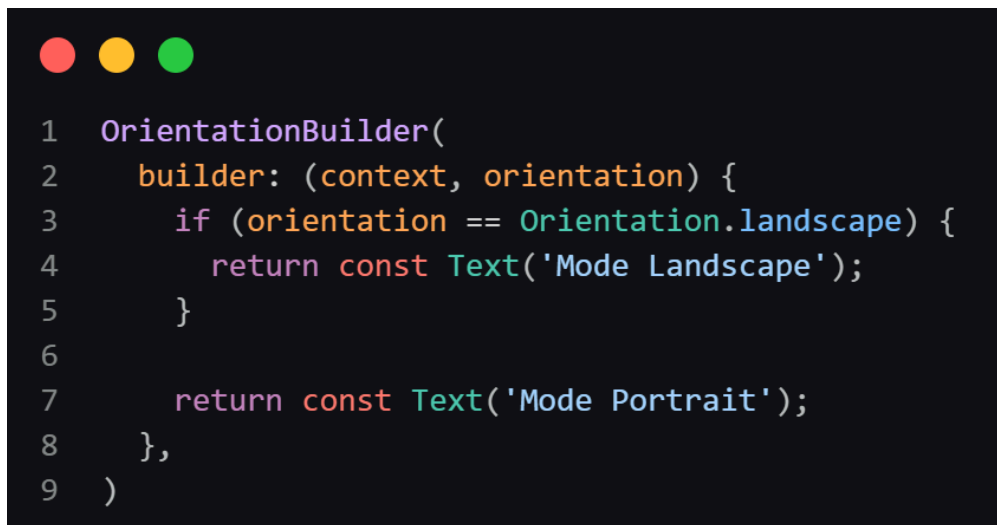


```
1 final size = MediaQuery.of(context).size;
2 final width = size.width;
3 final height = size.height;
4
5 Text('Lebar: ${width.toStringAsFixed(0)}'),
6 Text('Tinggi: ${height.toStringAsFixed(0)}'),
```

Informasi ukuran dapat digunakan untuk menyesuaikan padding, ukuran teks, atau komposisi elemen pada halaman.

4. Orientation

Orientation menunjukkan apakah ruang layar sedang digunakan dalam kondisi portrait atau landscape. Flutter menyediakan OrientationBuilder untuk membangun UI berdasarkan orientation layar.



```
1 OrientationBuilder(  
2   builder: (context, orientation) {  
3     if (orientation == Orientation.landscape) {  
4       return const Text('Mode Landscape');  
5     }  
6  
7     return const Text('Mode Portrait');  
8   },  
9 )
```

Uji aplikasi dengan mengubah orientasi emulator atau perangkat untuk melihat perubahan tampilan.

5. Constraints

Constraints adalah batasan ruang yang diberikan oleh widget parent kepada widget child. Batasan ini menentukan ruang yang dapat digunakan oleh widget child. Dalam Flutter, beberapa widget dapat digunakan untuk mengatur penggunaan ruang:

- Gunakan Expanded atau Flexible ketika child perlu berbagi ruang dalam Row atau Column.
- Gunakan SizedBox atau Padding untuk memberikan ukuran atau jarak yang terkontrol.

- Gunakan `LayoutBuilder` ketika keputusan layout bergantung pada ruang yang tersedia dari parent.

6. `LayoutBuilder`

`LayoutBuilder` digunakan untuk mengetahui ruang yang tersedia pada widget sehingga kita dapat menentukan layout yang sesuai.

```
1  LayoutBuilder(  
2    builder: (context, constraints) {  
3      final isWide = constraints.maxWidth >= 600;  
4  
5      return isWide  
6        ? const Row(  
7          children: [  
8            Expanded(child: ProfileHeader()),  
9            Expanded(child: ProfileInfo()),  
10         ],  
11       )  
12      : const Column(  
13        children: [  
14          ProfileHeader(),  
15          ProfileInfo(),  
16        ],  
17      );  
18    },  
19  )
```

`MediaQuery` membaca kondisi layar/media, sedangkan `LayoutBuilder` membaca constraints yang diterima widget dari parent.

7. Material Design

Material Design digunakan sebagai pendekatan desain antarmuka yang menyediakan komponen dan pola interaksi yang konsisten. Flutter menyediakan banyak komponen Material melalui package `material.dart`.

- Scaffold untuk struktur halaman.

- AppBar untuk bagian atas halaman.
- Card untuk mengelompokkan informasi.
- FilledButton atau ElevatedButton untuk aksi pengguna.
- Switch atau SwitchListTile untuk mengubah pengaturan.

8. ThemeData

ThemeData digunakan untuk mendefinisikan tampilan global aplikasi, seperti skema warna, tipografi, dan gaya komponen.



```

1  ThemeData(
2    useMaterial3: true,
3    colorScheme: ColorScheme.fromSeed(
4      seedColor: Colors.indigo,
5      brightness: Brightness.light,
6    ),
7  )

```

9. ColorScheme

ColorScheme digunakan untuk mengatur warna utama aplikasi agar tampilan tetap konsisten.



```

1  final lightTheme = ThemeData(
2    useMaterial3: true,
3    colorScheme: ColorScheme.fromSeed(
4      seedColor: Colors.indigo,
5      brightness: Brightness.light,
6    ),
7  );
8
9  final darkTheme = ThemeData(
10   useMaterial3: true,
11   colorScheme: ColorScheme.fromSeed(
12     seedColor: Colors.indigo,
13     brightness: Brightness.dark,
14   ),
15 );

```


10. Typography

Typography mengatur hierarki dan gaya teks pada aplikasi. Flutter menyediakan TextTheme yang dapat digunakan agar ukuran dan gaya teks konsisten.

```
1 Text(  
2   'Nama Mahasiswa',  
3   style: Theme.of(context).textTheme.headlineSmall,  
4 )  
5  
6 Text(  
7   'Ilmu Komputer',  
8   style: Theme.of(context).textTheme.bodyLarge,  
9 )
```

11. Light Theme dan Dark Theme

Flutter dapat menyediakan tema terang dan tema gelap. Perubahan tema dapat dikelola menggunakan ThemeMode pada MaterialApp.

```
1 MaterialApp(  
2   theme: lightTheme,  
3   darkTheme: darkTheme,  
4   themeMode: themeMode,  
5   home: const ResponsiveProfilePage(),  
6 )
```

PRAKTIKUM

1. Membuat Branch Pertemuan 5

Buat branch untuk pertemuan 5 dengan nama branch pertemuan-5

```
• PS D:\SEMESTER 7\MOLA\Prak Mola\prakmola_bintang> git checkout -b pertemuan-5  
Switched to a new branch 'pertemuan-5'
```

Pastikan branch aktif yang ditampilkan adalah pertemuan-5.

2. Membuat Konfigurasi Theme

Buat file lib/app_theme.dart. File ini digunakan untuk menyimpan Light Theme dan Dark Theme.

```
1  import 'package:flutter/material.dart';  
2  
3  class AppTheme {  
4      static final ThemeData lightTheme = ThemeData(  
5          useMaterial3: true,  
6          colorScheme: ColorScheme.fromSeed(  
7              seedColor: Colors.indigo,  
8              brightness: Brightness.light,  
9          ),  
10     );  
11  
12     static final ThemeData darkTheme = ThemeData(  
13         useMaterial3: true,  
14         colorScheme: ColorScheme.fromSeed(  
15             seedColor: Colors.indigo,  
16             brightness: Brightness.dark,  
17         ),  
18     );  
19 }
```

3. Membuat Halaman Responsive

Selanjutnya buat file: lib/responsive_profile.dart. File ini digunakan untuk membuat halaman profile yang dapat menyesuaikan tampilan berdasarkan ukuran dan orientasi layar.

```

1  import 'package:flutter/material.dart';
2
3  class ResponsiveProfilePage extends StatelessWidget {
4    final VoidCallback onThemeChanged;
5
6    const ResponsiveProfilePage({
7      super.key,
8      required this.onThemeChanged,
9    });
10
11    @override
12    Widget build(BuildContext context) {
13      final size = MediaQuery.of(context).size;
14
15      return Scaffold(
16        appBar: AppBar(
17          title: const Text('Responsive Profile'),
18          actions: [
19            IconButton(
20              onPressed: onThemeChanged,
21              icon: const Icon(Icons.brightness_6),
22            ),
23          ],
24        ),
25        body: Padding(
26          padding: EdgeInsets.all(size.width < 600 ? 16 : 32),
27          child: LayoutBuilder(
28            builder: (context, constraints) {
29              return OrientationBuilder(
30                builder: (context, orientation) {
31                  final isWide =
32                    constraints.maxWidth >= 600 ||
33                    orientation == Orientation.landscape;
34
35                  return isWide
36                    ? const Row(
37                      crossAxisAlignment:
38                        CrossAxisAlignment.start,
39                      children: [
40                        Expanded(child: _ProfileHeader()),
41                        SizedBox(width: 24),
42                        Expanded(child: _ProfileInfo()),
43                      ],
44                    )
45                    : const Column(
46                      children: [
47                        _ProfileHeader(),
48                        SizedBox(height: 20),
49                        _ProfileInfo(),
50                      ],
51                    );
52                },
53              );
54            },
55          ),
56        ),
57      );
58    }
59  }

```

Setelah halaman responsive dibuat, tambahkan widget `_ProfileHeader`.

```
1 class _ProfileHeader extends StatelessWidget {
2   const _ProfileHeader();
3
4   @override
5   Widget build(BuildContext context) {
6     return Card(
7       child: Padding(
8         padding: const EdgeInsets.all(24),
9         child: Column(
10          children: [
11            CircleAvatar(
12              radius: 48,
13              child: Icon(
14                Icons.person,
15                size: 48,
16                color: Theme.of(context).colorScheme.primary,
17              ),
18            ),
19            const SizedBox(height: 16),
20            Text(
21              'Nama Mahasiswa',
22              style: Theme.of(context).textTheme.headlineSmall,
23              textAlign: TextAlign.center,
24            ),
25          ],
26        ),
27      ),
28    );
29  }
30 }
```

Selanjutnya tambahkan `_ProfileInfo`.

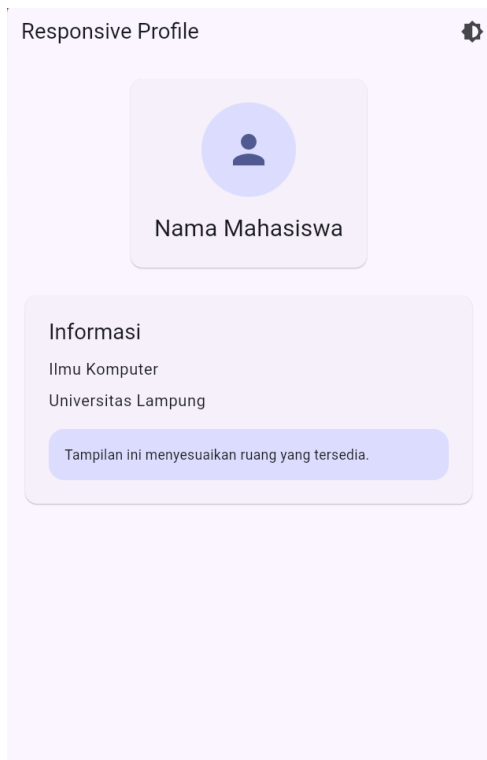
```
1 class _ProfileInfo extends StatelessWidget {
2   const _ProfileInfo();
3
4   @override
5   Widget build(BuildContext context) {
6     final scheme = Theme.of(context).colorScheme;
7
8     return Card(
9       child: Padding(
10        padding: const EdgeInsets.all(24),
11        child: Column(
12          crossAxisAlignment: CrossAxisAlignment.start,
13          children: [
14            Text(
15              'Informasi',
16              style: Theme.of(context).textTheme.titleLarge,
17            ),
18            const SizedBox(height: 12),
19            Text(
20              'Ilmu Komputer',
21              style: Theme.of(context).textTheme.bodyLarge,
22            ),
23            const SizedBox(height: 8),
24            Text(
25              'Universitas Lampung',
26              style: Theme.of(context).textTheme.bodyLarge,
27            ),
28            const SizedBox(height: 16),
29            Container(
30              width: double.infinity,
31              padding: const EdgeInsets.all(16),
32              decoration: BoxDecoration(
33                color: scheme.primaryContainer,
34                borderRadius: BorderRadius.circular(16),
35              ),
36              child: Text(
37                'Tampilan ini menyesuaikan ruang yang tersedia.',
38                style: Theme.of(context).textTheme.bodyMedium,
39              ),
40            ),
41          ],
42        ),
43      ),
44    );
45  }
46 }
```

4. Mengatur main.dart

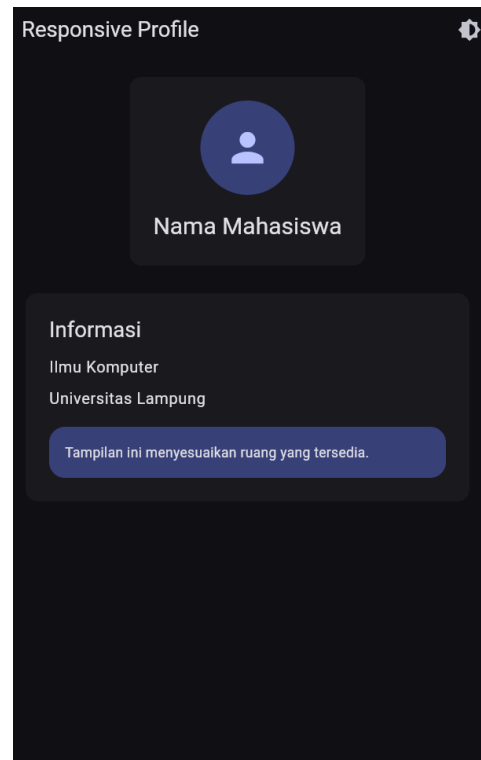
Ubah file main.dart untuk menghubungkan Light Theme, Dark Theme, ThemeMode, dan halaman responsive.

```
1  import 'package:flutter/material.dart';
2  // import 'column_widget.dart';
3  // import 'row_widget.dart';
4  // import 'first_widget.dart';
5  // import 'form_widget.dart';
6  import 'app_theme.dart';
7  import 'responsive_profile.dart';
8
9  void main() {
10   runApp(const MyApp());
11 }
12
13 class MyApp extends StatefulWidget {
14   const MyApp({super.key});
15
16   @override
17   State<MyApp> createState() => _MyAppState();
18 }
19
20 class _MyAppState extends State<MyApp> {
21   ThemeMode themeMode = ThemeMode.light;
22
23   void toggleTheme() {
24     setState(() {
25       themeMode = themeMode == ThemeMode.light
26         ? ThemeMode.dark
27         : ThemeMode.light;
28     });
29   }
30
31   @override
32   Widget build(BuildContext context) {
33     return MaterialApp(
34       debugShowCheckedModeBanner: false,
35       title: 'Responsive Profile',
36       theme: AppTheme.lightTheme,
37       darkTheme: AppTheme.darkTheme,
38       themeMode: themeMode,
39       home: ResponsiveProfilePage(
40         onThemeChanged: toggleTheme,
41       ),
42     );
43   }
44 }
```

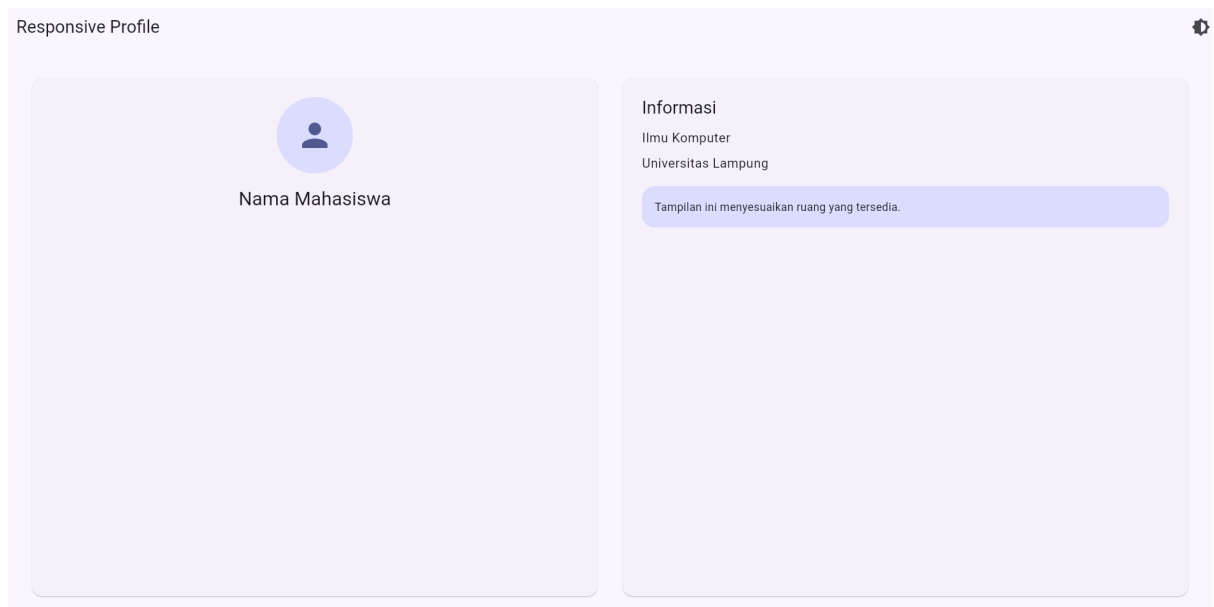
5. Menjalankan Aplikasi



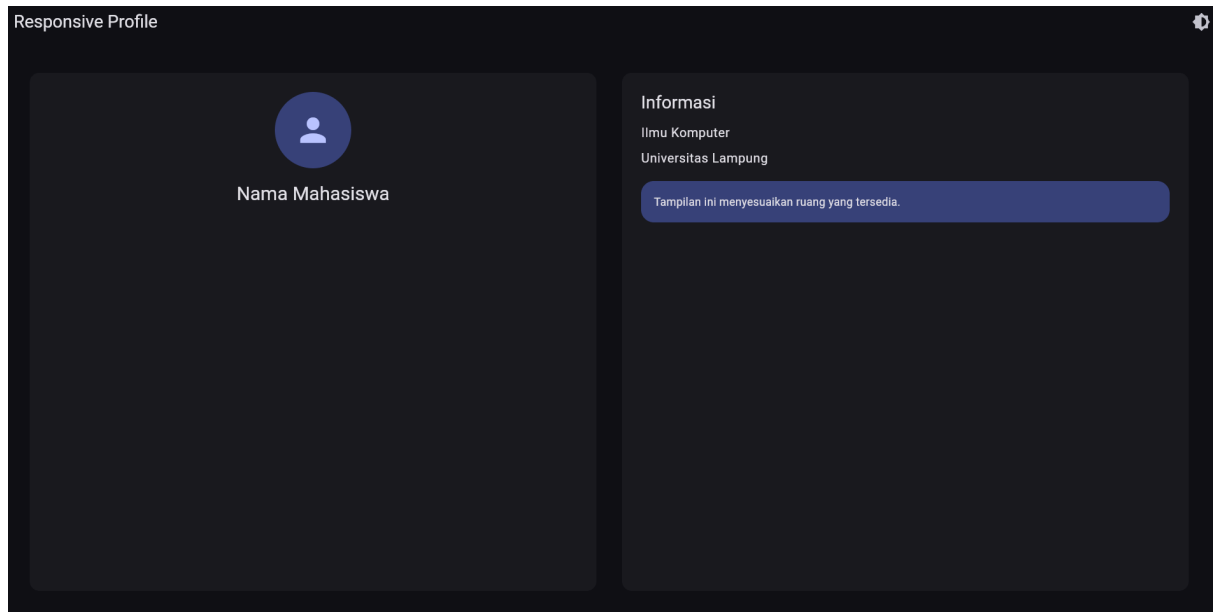
Portrait — Light Mode



Portrait — Dark Mode



Landscape — Light Mode



Landscape — Dark Mode

6. Push File Terbaru ke GitHub

- Tambahkan seluruh perubahan file terbaru pada project dengan perintah git add .

```
● PS D:\SEMESTER 7\MOLA\Prak Mola\prakmola_bintang> git add .
```

- Berikan perintah git commit -m "Isi pesan" untuk menambahkan pesan file yang ingin di commit

```
● PS D:\SEMESTER 7\MOLA\Prak Mola\prakmola_bintang> git commit -m "Pertemuan 5: Adaptive Responsive Design and Theming"
```

- Masukkan perintah git push origin pertemuan-5 untuk melakukan push project ke branch github yang terbaru

```
● PS D:\SEMESTER 7\MOLA\Prak Mola\prakmola_bintang> git push origin pertemuan-5
```

- Buka GitHub, Lakukan Compare & pull request pada repository
- Create a pull request untuk membuat request dari branch terbaru
- Lalu Klik Tombol Merge Pull Request
- Maka perubahan yang dilakukan telah berhasil dilakukan.

TUGAS MAHASISWA

- Mahasiswa mengikuti tahapan pembuatan aplikasi Responsive Profile diatas, dapat dilihat dari modul ataupun dari video penjelasan yang diberikan.
- Jika sudah mahasiswa dapat merekam lalu menjelaskan alur pembuatannya dari awal hingga selesai, dengan mengikuti **point 2 sampai 5** diatas, waktu rekaman video maksimal 5 menit saja.
- Mahasiswa dapat upload hasil rekaman ke YouTube, dengan format judul video: **NPM_NAMA_Tugas02**
- Kumpulkan dalam bentuk **link video YouTube** pada GForm yang disediakan.
- Link: